Laporan Praktikum
Praktikum Pengujian Perangkat Lunak
Pertemuan 1
Unit Test & Assertion

Nama Mahasiswa: Januarsyah Akbar (24/535846/SV/24314)
Kelas B2
Dosen Pengampu: Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

February 10, 2026

Contents

1 Tujuan Praktikum **2**

2 Langkah Praktikum **3**
2.1 Unit Test 3
2.2 Assertion 5

3 Tugas Praktikum **8**
3.1 Unit Test 8

4 Kesimpulan **10**

Chapter 1

Tujuan Praktikum

1. Mahasiswa mampu memahami unit testing
2. Mahasiswa mampu menggunakan JUnit
3. Mahasiswa memahami konsep assertion
4. Mahasiswa mampu mengimplementasikan assertion pada pengujian unit

Chapter 2

Langkah Praktikum

2.1 Unit Test

1. Buka IDE yang digunakan, seperti IntelliJ IDEA atau Eclipse.

2. Buat proyek Java baru dengan Build system Maven (pada laporan ini menggunakan IntelliJ IDEA, dengan JDK versi 23).

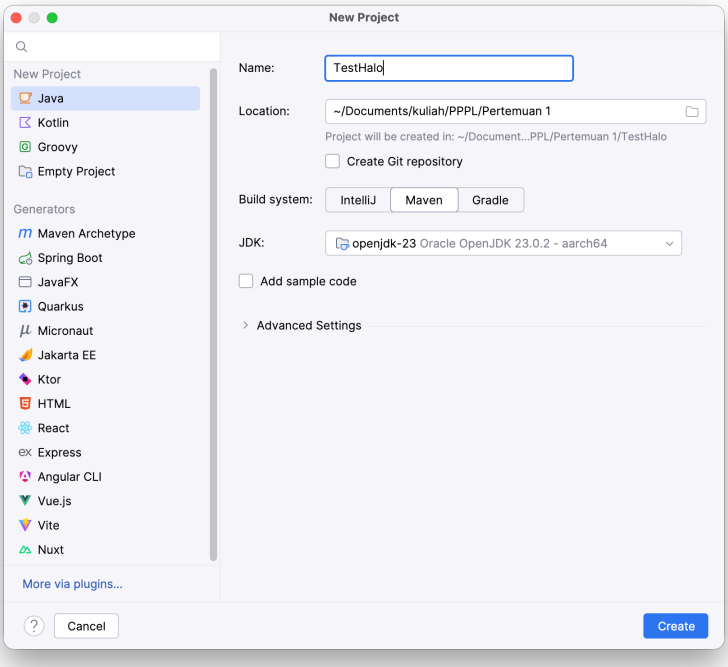

Figure 2.1: Membuat Proyek Baru di IntelliJ IDEA

3. Tambahkan dependensi JUnit pada file `pom.xml` untuk menggunakan framework JUnit dalam proyek. Kunjungi halaman berikut untuk menambahkan JUnit secara manual:

   `https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter/5.13.4`

   Pilih Maven dengan Scope Test dan salin kode dependensi yang diberikan. Paste kodet

tadi ke dalam tag <dependencies> di file `pom.xml` seperti berikut:

```
<dependencies>
<!-- Source: https://mvnrepository.com/artifact/org.junit.jupiter/junit-
    jupiter -->
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.13.4</version>
    <scope>test</scope>
</dependency>
</dependencies>
```

Listing 2.1: Menambahkan Dependensi JUnit di pom.xml

4. Setelah menambahkan dependensi jalankan shortcut Command + Shift + I untuk sync Maven.

5. Buat class biasa, contoh return "hello world" pada class tersebut.

```
// location: src/main/java/Hello.java
public class Hello {
    public String helloWorld()
    {
        return "hello world";
    }
}
```

Listing 2.2: Contoh Class Java Sederhana

6. Buat class test pada folder `src/test/java` dengan nama yang sama dengan class yang akan di test, namun dengan tambahan suffix `Test`.

```
// location: src/test/java/HelloTest.java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
public class HelloTest {
    @Test
    public void testHelloWorld() {
        Hello hello = new Hello();
        String result = hello.helloWorld();
        assertEquals("hello world", result);
    }
}
```

Listing 2.3: Contoh Class Test Java Sederhana

7. Jalankan class test untuk memastikan bahwa unit test berjalan dengan benar.

8. Jika semua langkah di atas dilakukan dengan benar, maka hasil dari unit test akan menunjukkan bahwa semua test berhasil (green bar) seperti gambar berikut:

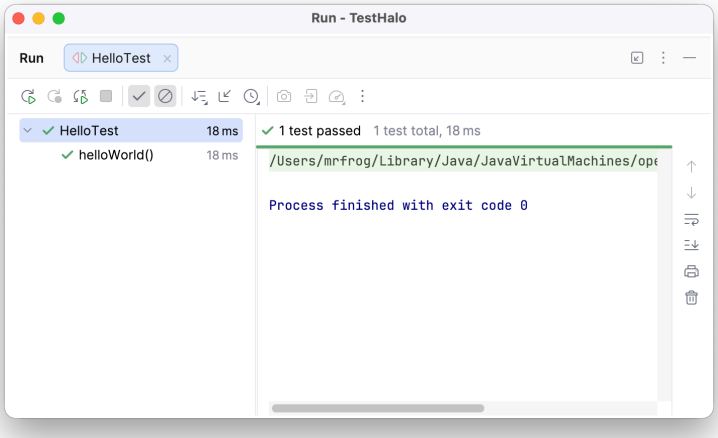

Figure 2.2: Hasil Unit Test Berhasil

2.2 Assertion

1. Buat class Java baru dengan nama `Calculator.java` pada folder `src/main/java` dengan isi sebagai berikut:

```java
public class Calculator {
    int a, b;

    public Calculator(int a, int b) {
        this.a = a;
        this.b = b;
    }

    public int tambah() {
        return a + b;
    }

    public int kurang() {
        return a - b;
    }

    public int bagi() {
        return a / b;
    }

    public int kali() {
        return a * b;
    }
}
```

Listing 2.4: Contoh Class Calculator Java Sederhana

2. Klik kanan pada nama class dan pilih show context actions, lalu pilih `Create Test` untuk membuat class test baru.

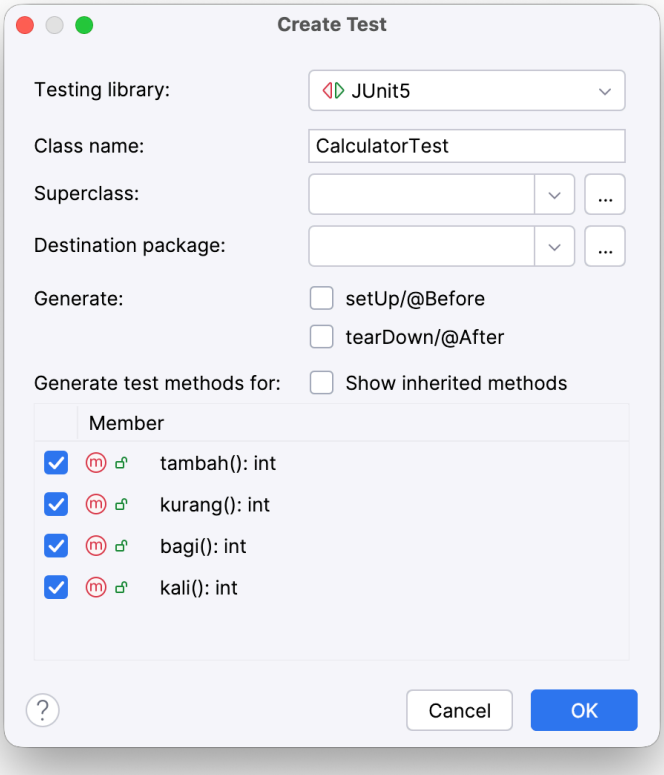

Figure 2.3: Membuat Class Test dengan Context Actions

3. Pilih metode yang akan di test, lalu klik OK.

4. Maka akan terbentuk class test baru, edit class test tersebut untuk menambahkan assertion pada setiap metode yang di test seperti berikut:

```java
// location: src/test/java/CalculatorTest.java
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

    @Test
    void tambah() {
        Calculator Cal =  new Calculator(2,5);
        int actual = Cal.tambah();
        int expected = 7;

        assertEquals(actual,expected);
    }
```

Listing 2.5: Contoh Class Test Calculator Java Sederhana

```java
    @Test
    void kurang() {
        Calculator Cal = new Calculator(5, 2);
        int actual = Cal.kurang();
        int expected = 3;

        assertEquals(actual, expected);
    }

    @Test
    void bagi() {
        Calculator Cal = new Calculator(10, 2);
        int actual = Cal.bagi();
        int expected = 5;

        assertEquals(actual, expected);
    }

    @Test
    void kali() {
        Calculator Cal = new Calculator(3, 4);
        int actual = Cal.kali();
        int expected = 12;

        assertEquals(actual, expected);
    }
}
```

Listing 2.6: Contoh Class Test Calculator Java Sederhana

5. Jalankan class test untuk memastikan bahwa semua metode pada class `Calculator` telah diuji dengan benar.

6. Jika semua langkah di atas dilakukan dengan benar, maka hasil dari unit test akan menunjukkan bahwa semua test berhasil (green bar) seperti gambar berikut:

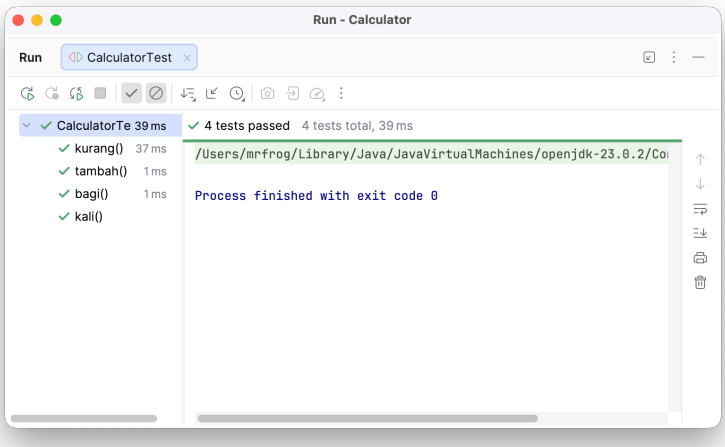

Figure 2.4: Hasil Unit Test Calculator Berhasil

Chapter 3

Tugas Praktikum

3.1 Unit Test

1. Buatlah sebuah kelas uji/test class yang berisi 2 test method. Method pertama menampilkan pesan "test class method 1" dan method ke-2 menampilkan pesan "test class method 2".

```java
// location: src/test/java/MyTestClass.java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class MyTestClass {
    @Test
    public void testMethod1() {
        System.out.println("test class method 1");
        assertTrue(true);
    }

    @Test
    public void testMethod2() {
        System.out.println("test class method 2");
        assertTrue(true);
    }
}
```

Listing 3.1: Test Class dengan 2 Test Method

2. Jalankan test class yang telah dibuat. berikut adalah hasil eksekusi dari test class tersebut:

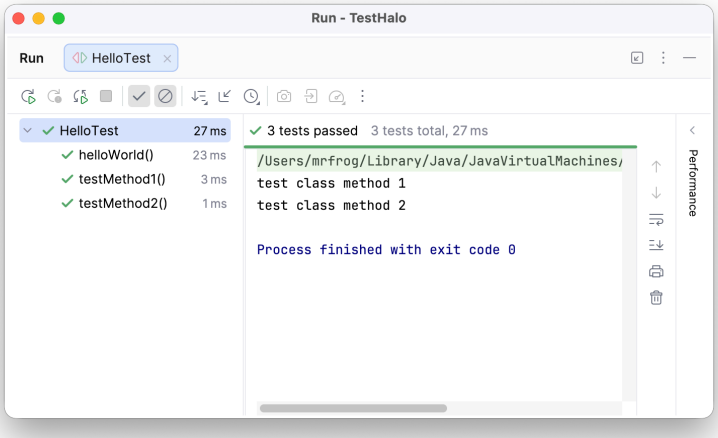

Figure 3.1: Hasil Eksekusi Test Class

Hasil: Kedua test method berhasil dijalankan dengan ditandai oleh green bar pada IDE. Output console menampilkan pesan dari masing-masing test method sesuai dengan yang diharapkan.

Chapter 4

Kesimpulan